

Practical Number		CO/PO
	Process: withdraw() finds the balance is insufficient and throws InsufficientFundsException with shortfall = 4000; the catch block in main handles it. Expected Output: The program does not crash; it prints a message such as “Withdrawal failed: short by 4000” and the balance stays 1000.	

Practical Number		CO/PO
9	Part A — Practice Programs (Hour 1) Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run. 2. Seat booking race: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do. 3. (For Advanced Learners – home) Split a large array of numbers across several threads that add into one shared total, show the wrong result, then fix it two ways and compare; decide how to measure the difference. Part B — Project Work (Hour 2): MiniBank A real bank serves many users at once. With the single-user logic working, this lab runs many threads against one account, lets you watch the balance come out wrong (a race condition), and then makes deposit and withdraw thread-safe with synchronization — protecting MiniBank’s shared data. 1. Create a class named AccountWorker that implements Runnable with fields Account account, int times and long amount; in run(), loop times times and call account.deposit(amount). 2. In main, create one account, start about 10 threads using AccountWorker, call join() on each, then print the final balance — it will be wrong because of the race condition. 3. Make deposit and withdraw synchronized (or use a synchronized block on a private lock object) and run the program again — the balance is now correct. 4. Print the thread names to observe the thread lifecycle. Key Questions / Analysis / Interpretation to be evaluated during/after Implementation 1. What is a race condition, and why does it occur here? 2. What does the synchronized keyword do? 3. What are the states in a thread’s life cycle? Supplementary Problems – • Replace synchronized with an AtomicLong balance and compare the results. • Add withdrawing threads as well and confirm the balance stays correct. Key Skills to be addressed – Thread and Runnable; starting and joining threads; the thread life cycle; race conditions; synchronized methods and blocks; thread safety. Applications – Any server handling many users at once — banking, ticketing, inventory — must make shared data thread-safe. Learning Outcome – The student can create threads and protect shared data using synchronization. (CO4) Dataset / Test Data (Source and Description, if applicable): No external dataset. The threads and their counts are set in code (e.g. 10 threads, 1000 deposits of 1 each). Tools / Technology To Be Used: JDK 17+, GitHub Codespaces (or a local IDE), command line, git. • Total Hours of Problem Definition Implementation: 2 hours — Hour 1: Part A practice programs; Hour 2: Part B project work. • Total Hours of Engagement = time to implement the solution + modification time (if necessary) + testing by the faculty member: approximately 2.5 hours.	CO4 PO5, PSO1

Practical Number		CO/PO
	Post Laboratory Work Description: Commit and tag lab-09. Record the wrong balance before synchronization and the correct balance after, and explain the difference. Rubrics (Practical Evaluation / Viva): Part A practice programs – 20%. Part B project (80%): Working multithreaded deposits – 30%; demonstrating the race condition – 25%; correct synchronization fix – 30%; viva – 15%. Expected Input / Process / Output Input: In code: 10 threads, each calling deposit(1) one thousand times, on one account starting at balance 0. Process: Without synchronization the threads overwrite each other's updates; with synchronization each update completes before the next begins. Expected Output: Before the fix the printed balance is less than 10000 and varies each run; after the fix it is exactly 10000 every run.	

Practical Number		CO/PO
10	Part A — Practice Programs (Hour 1) Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home. 1. Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused. 2. Producer-consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost. 3. (For Advanced Learners – home) Reproduce a deadlock between two threads that lock two objects in opposite order, then remove it with a consistent lock order. Choose the scenario yourself. Part B — Project Work (Hour 2): MiniBank Building on basic threads, this lab gives MiniBank a proper transaction engine. You process a batch of transactions on a managed thread pool, coordinate two threads with wait/notify, and learn to avoid deadlock with consistent lock ordering — the way back-office systems process many transactions safely. 1. Create a class named TransactionProcessor that holds an ExecutorService created with Executors.newFixedThreadPool(4). Add a method submit(Runnable task) that calls execute(), and a method stop() that calls shutdown() and awaitTermination(). 2. Create Runnable tasks that each perform a deposit or withdrawal; submit many of them, then stop the processor and wait for completion. 3. Build a simple producer-consumer: one thread adds transactions to a shared array buffer and another thread processes them, using wait() and notify() inside synchronized blocks. 4. Create a deadlock: two threads transfer money A to B and B to A, locking the two accounts in opposite order. Then fix it by always locking the account with the smaller accountNumber first. Key Questions / Analysis / Interpretation to be evaluated during/after Implementation 1. Why is a thread pool better than creating a new thread per task? 2. How do wait() and notify() coordinate two threads? 3. What causes a deadlock, and how does consistent lock ordering prevent it? Supplementary Problems – • Print how many tasks each pool thread executed. • Detect a long-running transfer and log a warning. Key Skills to be addressed – ExecutorService and thread pools; submitting Runnable tasks; inter-thread communication with wait/notify; deadlock and lock ordering. Applications – Batch processors, web-server request pools and job-scheduling systems all use managed thread pools.	CO4 PO5, PO11, PSO1